Approximating the Average-Case Graph Search Problem with Non-Uniform Costs

Michał Szyfelbein

Gdańsk University of Technology, Poland

March 5, WALCOM 2026

Binary Search

Binary Search – a classical strategy used to efficiently locate a hidden **target** element x in a linearly ordered set using $O(\log n)$ comparison operations.

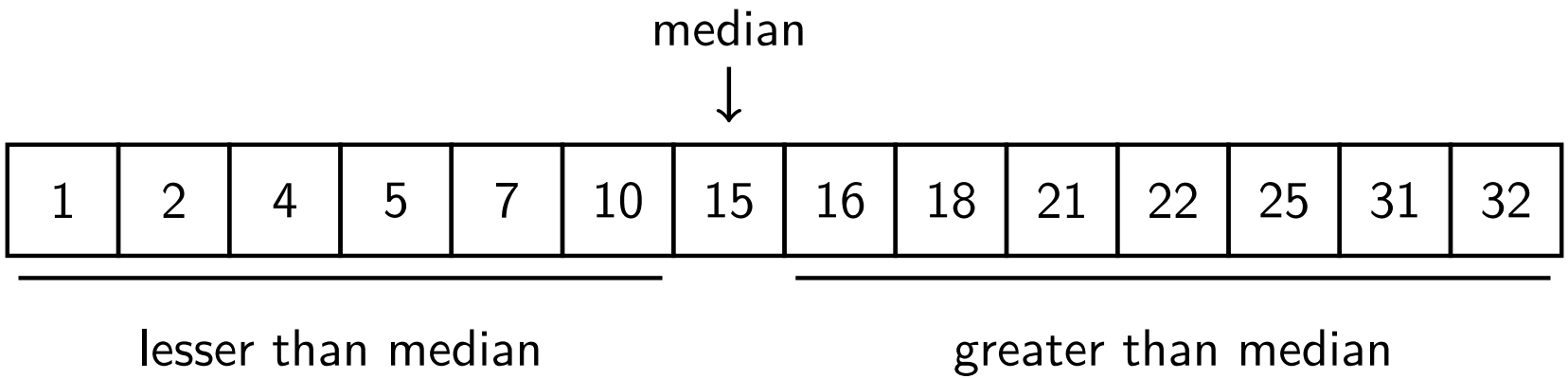


Figure: Example of a sorted array containing 14 elements.

Searching in Graphs

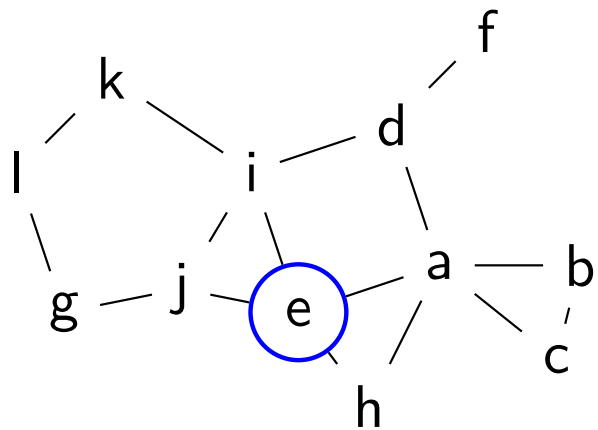
Easy to generalize to graphs:

A **query** to a vertex v returns information whether v is the target x, and if not, which connected component of $G - v$ contains x.

Searching in Graphs

Easy to generalize to graphs:

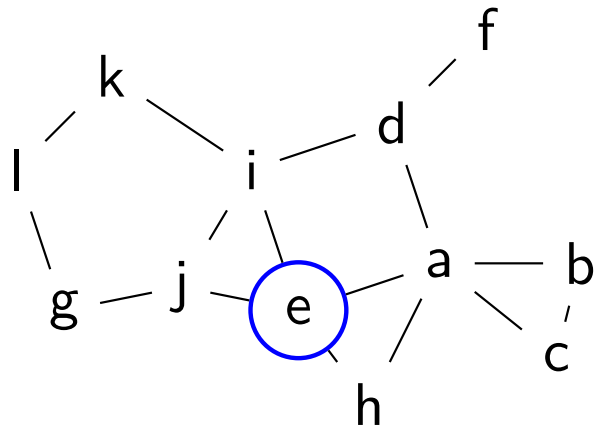
A **query** to a vertex v returns information whether v is the target x, and if not, which connected component of $G - v$ contains x.

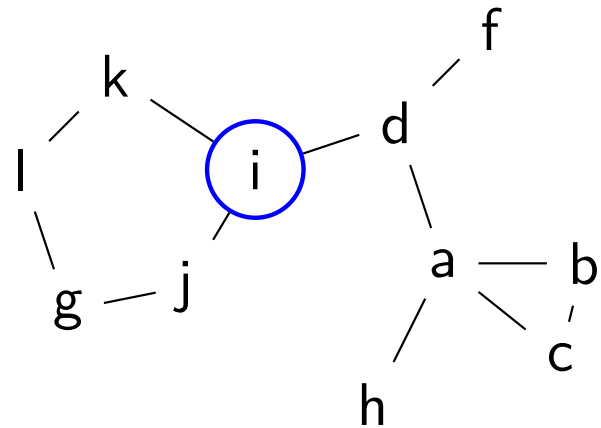


Searching in Graphs

Easy to generalize to graphs:

A **query** to a vertex v returns information whether v is the target x, and if not, which connected component of $G - v$ contains x.

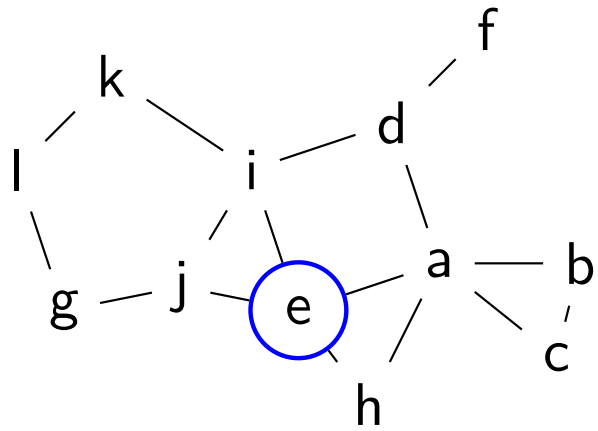


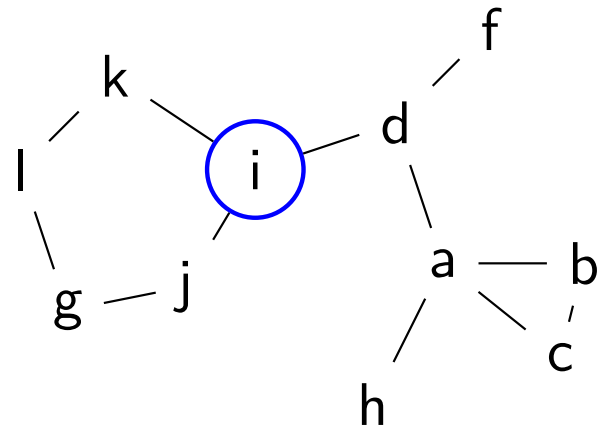


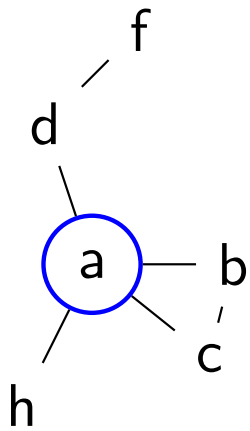
Searching in Graphs

Easy to generalize to graphs:

A **query** to a vertex v returns information whether v is the target x, and if not, which connected component of $G - v$ contains x.



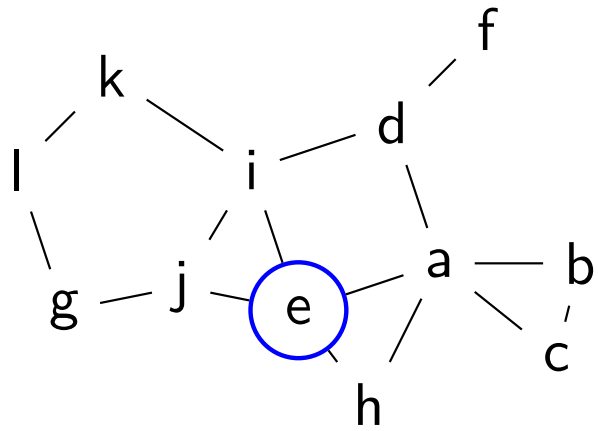


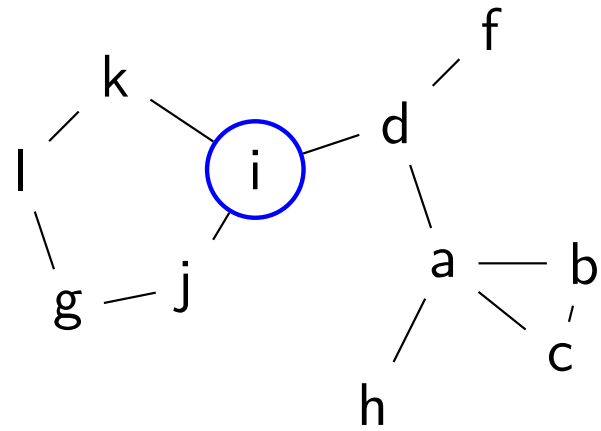


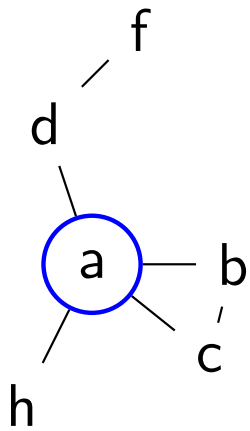
Searching in Graphs

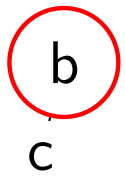
Easy to generalize to graphs:

A **query** to a vertex v returns information whether v is the target x, and if not, which connected component of $G - v$ contains x.









Strategy of searching

We wish to find a strategy of searching.

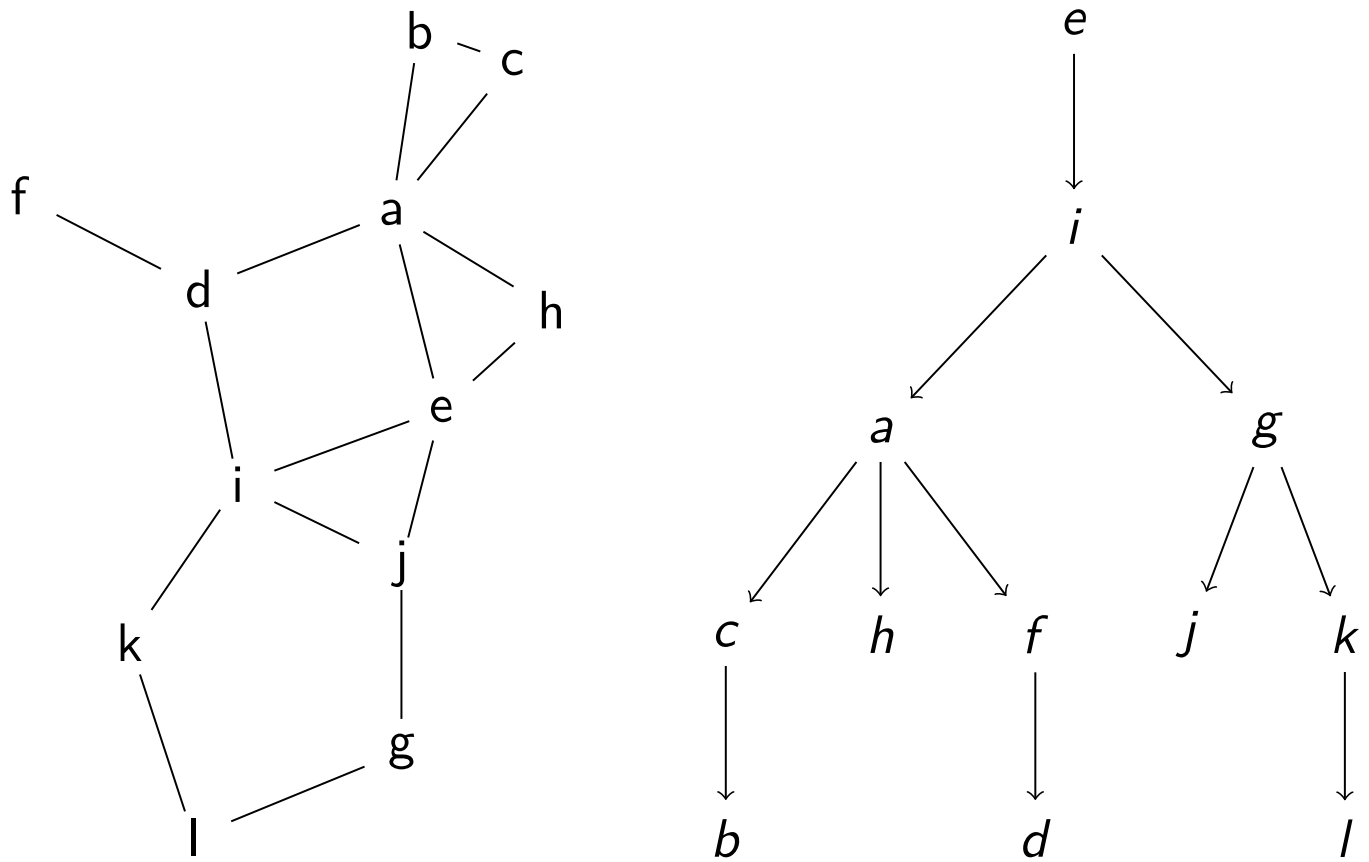


Figure: Sample input graph a decision tree for it.

Additional parameters

We introduce additional information about the input:

- To each vertex v we assign an arbitrary **cost** $c(v)$, which denotes a cost of performing a query to v.
- Additionally, to each vertex v we also assign an arbitrary **weight** $w(v)$, which denotes the importance of v.

Our setup

What is the best strategy of searching in a graph?

Graph Search Problem (GSP)

Input: Graph G, a query cost function $c\colon V(G) \to \mathbb{N}$ and a weight function $w\colon V(G) \to \mathbb{N}$.

Output: A decision tree D minimizing the weighted average search cost:

$$c_G(D) = \sum_{x \in V(G)} w(x) \cdot c(Q_G(D, x))$$

where $Q_G(D, x)$ denotes the set of queries performed along the unique path in D from the root $r(D)$ to x.

Cost of searching

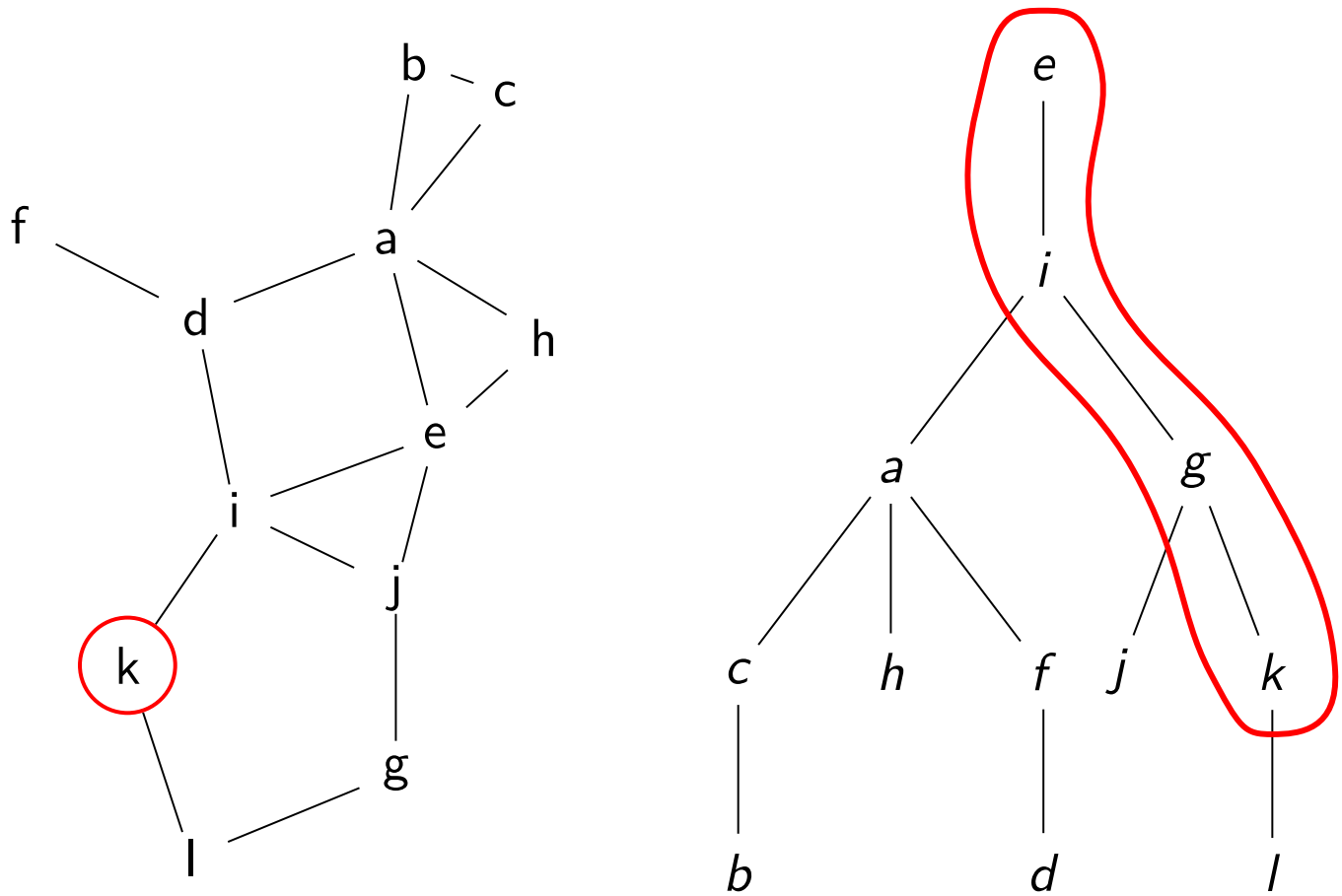


Figure: Vertices contributing their cost to the search time of k.

Our results

This problem is **NP–hard** already for trees. We give the following **approximation** algorithms:

- $(4+\epsilon)$–approximation for **trees**.

Our results

This problem is **NP–hard** already for trees. We give the following **approximation** algorithms:

- $(4+\epsilon)$–approximation for **trees**.
- $O(f_n)$-approximation algorithm, where f_n is the approximation ratio of any polynomial time algorithm for the Min–Ratio Vertex Cut Problem. In particular, this yields:

Our results

This problem is **NP–hard** already for trees. We give the following **approximation** algorithms:

- $(4+\epsilon)$–approximation for **trees**.
- $O(f_n)$-approximation algorithm, where f_n is the approximation ratio of any polynomial time algorithm for the Min–Ratio Vertex Cut Problem. In particular, this yields:
  - $O\left(\sqrt{\log n}\right)$–approximation for **general graphs**.
  - $O\left(|V(H)|^2\right)$-approximation for graphs excluding H as a minor which yields constant factor approximation for many important graph classes, such as planar graphs, graphs of bounded treewidth, etc.

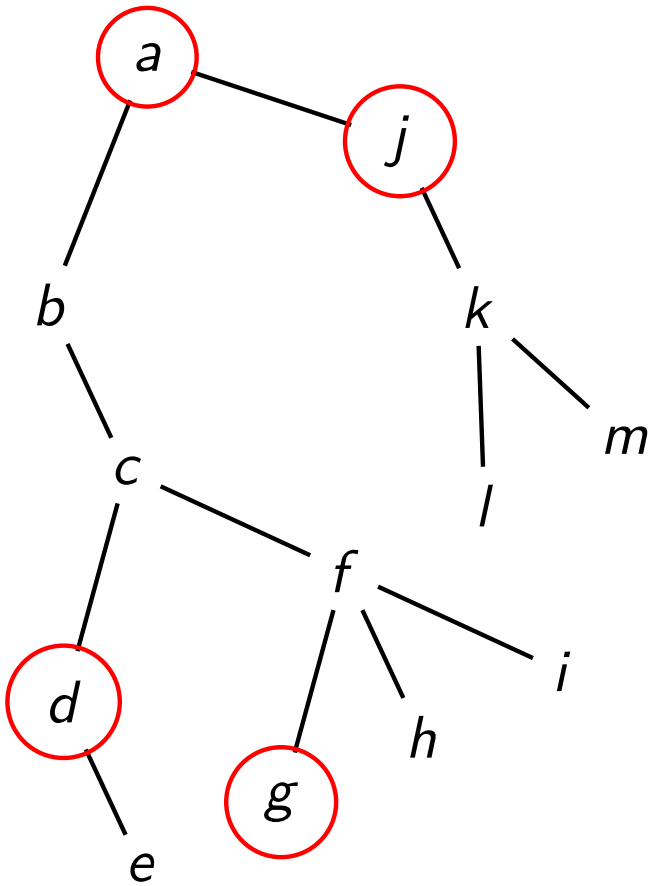
Weighted α–Separator Problem

Weighted α–Separator Problem

Input: Graph G, a cost function $c\colon V \to \mathbb{N}$, a weight function $w\colon V \to \mathbb{N}$ and a real number α.

Output: A set $S \subseteq V(G)$ called **separator** such that for every $H \in G - S$, $w(H) \leq w(G)/\alpha$ and $c(S)$ is minimized.

Example of a separator



	$w(v)$	$c(v)$
a	2	3
b	1	4
c	3	6
d	2	2
e	4	1
f	0	3
g	1	1
h	4	3
i	2	3
j	5	2
k	1	2
l	2	3
m	3	4

Figure: Sample input tree T and a weighted 3-separator (the circled vertices) of cost 8.

How to find the separator

Bad news: The Weighted α–separator Problem is **NP–hard** even for stars. But there exists a biciriteria **FPTAS** for trees:

Theorem

Let S be an optimal weighted α–separator for (T, c, w, α). For any $\delta > 0$ there exists an algorithm `SeparatorFPTAS`*, which returns a separator S', such that:*

1. $c(S') \leq c(S)$.
2. $w(H) \leq \frac{(1+\delta)\cdot w(T)}{\alpha}$ *for every* $H \in T - S'$.
3. *The algorithm runs in* $O\left(n^3/\delta^2\right)$ *time.*

Notation

To connect searching and separating we need the following notation:

- $\mathcal{R}_D(G)$ – the family of all candidate subsets of D in G.
- D^* – optimal decision tree.
- $\mathcal{L}_k^*$ – the k–th **level** of $\texttt{OPT}(G)$: the subset of $\mathcal{R}_{D^*}(G)$ consisting of all maximal elements H of $\mathcal{R}_{D^*}(G)$ with $w(H) \leq k$.
- $S_k^* = V(G) - \mathcal{L}_k^*$ – vertices belonging to the separator at the level $\mathcal{L}_k^*$. S_k^* forms a weighted $w(G)/k$–separator of G.

Example of the connection

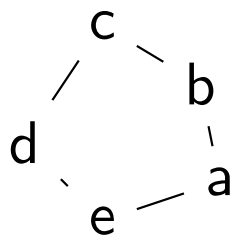


Figure: G.

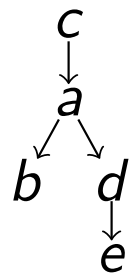


Figure: D^*.

Example of the connection

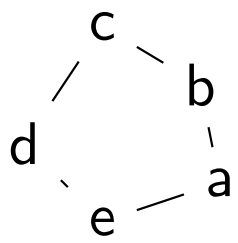


Figure: G.

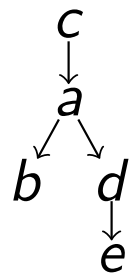


Figure: D^*.

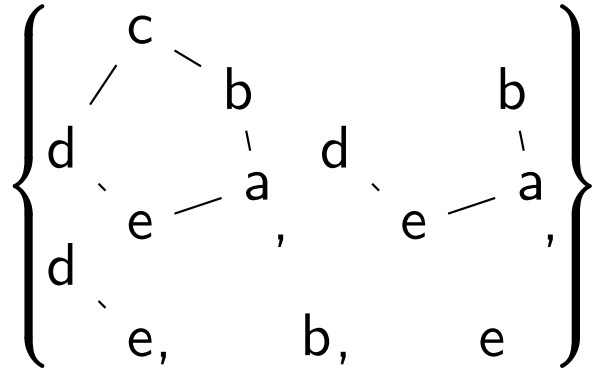


Figure: $\mathcal{R}_{D^*}(G)$.

Example of the connection

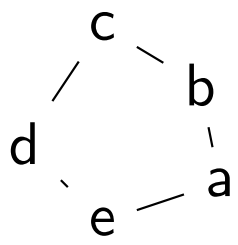


Figure: G.

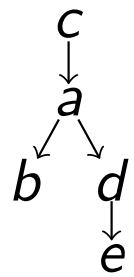


Figure: D^*.

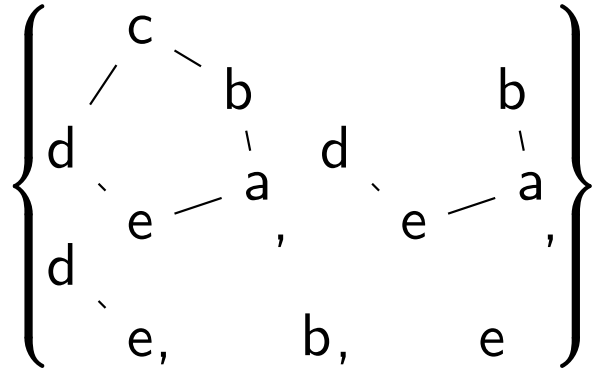


Figure: $\mathcal{R}_{D^*}(G)$.

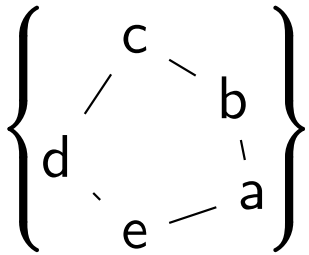


Figure: $\mathcal{L}_5^*, S_5^* = \{\}$.

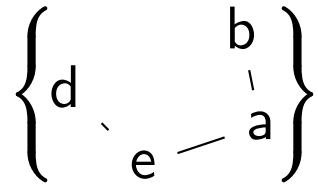


Figure: $\mathcal{L}_4^*, S_4^* = \{c\}$.

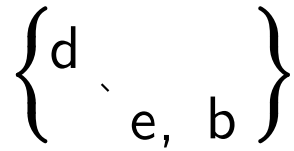


Figure: $\mathcal{L}_3^*, S_3^* = \{a, c\}$.

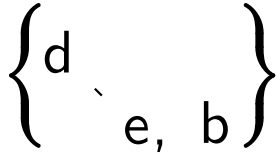


Figure: $\mathcal{L}_2^*, S_2^* = \{a, c\}$.

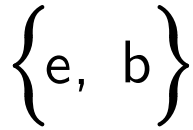


Figure: $\mathcal{L}_1^*, S_1^* = \{a, c, d\}$.

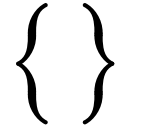


Figure: $\mathcal{L}_0^*, S_0^* = \{a, b, c, d, e\}$.

Basic lemmas

Lemma

Let $G_{D,v}$ be the candidate subgraph of G in which v is queried when using D. Then, $c_G(D) = \sum_{v \in V(G)} w(G_{D,v}) \cdot c(v)$.

Basic lemmas

Lemma

Let $G_{D,v}$ be the candidate subgraph of G in which v is queried when using D. Then, $c_G(D) = \sum_{v \in V(G)} w(G_{D,v}) \cdot c(v)$.

Lemma

$\mathit{OPT}(G) = \sum_{k=0}^{w(G)-1} c(S_k^*)$.

Basic lemmas

Lemma

$$2 \cdot \mathit{OPT}(G) = 2 \cdot \sum_{k=0}^{w(G)-1} c(S_k^*) \geq \sum_{k=0}^{w(G)} c\left(S^*_{\lfloor k/2 \rfloor}\right).$$

Basic lemmas

Lemma

$$2 \cdot \mathit{OPT}(G) = 2 \cdot \sum_{k=0}^{w(G)-1} c(S_k^*) \geq \sum_{k=0}^{w(G)} c\left(S^*_{\lfloor k/2 \rfloor}\right).$$

Lemma

Let $\mathcal{G}$ be any subgraph of G and $0 \leq \beta \leq 1$. Then:

$$\beta \cdot w(\mathcal{G}) \cdot c\left(S^*_{\lfloor w(\mathcal{G})/2 \rfloor} \cap \mathcal{G}\right) \leq \sum_{k=(1-\beta)w(\mathcal{G})+1}^{w(\mathcal{G})} c\left(S^*_{\lfloor k/2 \rfloor} \cap \mathcal{G}\right).$$

Searching in Trees

We will iteratively use the **FPTAS** for the Weighted α–separator problem to create an $(4+\epsilon)$–approximation algorithm for the Tree Search Problem:

Theorem

For any $\epsilon > 0$ there exists an $(4+\epsilon)$–approximation algorithm for the Tree Search Problem running in $O\left(n^4/\epsilon^2\right)$ time.

The algorithm

proc `DecisionTree`(T, c, w, ϵ):

1. $S_T \leftarrow$ `SeparatorFPTAS`$\left(T, c, w, \alpha = 2, \delta = \frac{\epsilon}{4+\epsilon}\right)$.
2. $D_T \leftarrow$ arbitrary partial decision tree for T, built from vertices of S_T.
3. For each $H \in T - S_T$:
   - 3.1 $D_H \leftarrow$ `DecisionTree`(H, c, w, ϵ).
   - 3.2 Hang D_H in D_T below the last query to $v \in N_T(H)$.
4. Return D_T.

Structure of the solution

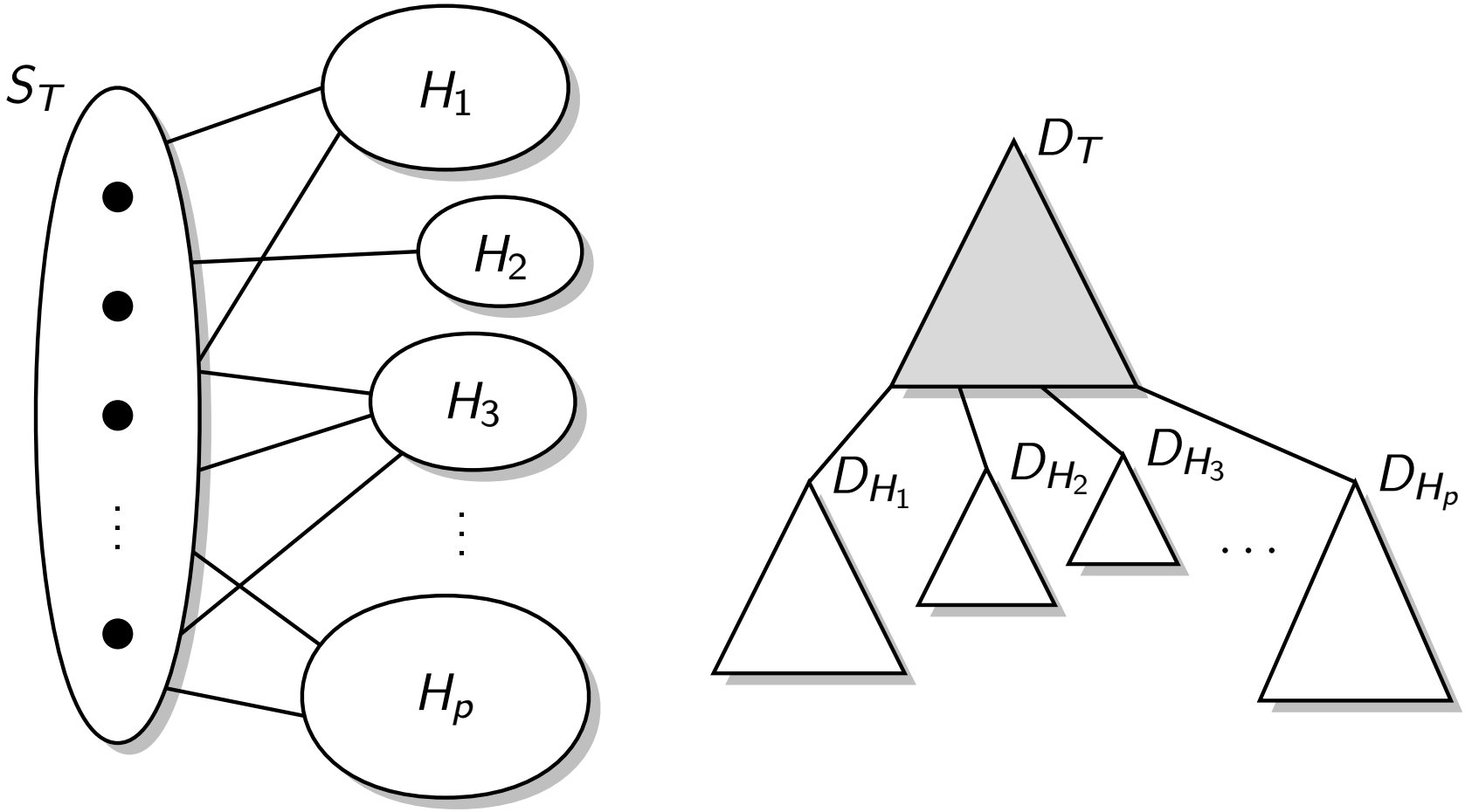


Figure: The separator S_T produced by the algorithm and the structure of the decision tree built using S_T.

Sketch of the proof

- $\mathcal{T}$ – subtree of T for which the procedure was called.
- Let $S^*_{\mathcal{T}} = S^*_{\lfloor w(\mathcal{T})/2 \rfloor} \cap \mathcal{T}$. We have $c(S_{\mathcal{T}}) \leq c(S^*_{\mathcal{T}})$.
- We have that the contribution of the decision tree $D_{\mathcal{T}}$ is bounded by:

Sketch of the proof

- $\mathcal{T}$ – subtree of T for which the procedure was called.
- Let $S^*_{\mathcal{T}} = S^*_{\lfloor w(\mathcal{T})/2 \rfloor} \cap \mathcal{T}$. We have $c(S_{\mathcal{T}}) \leq c(S^*_{\mathcal{T}})$.
- We have that the contribution of the decision tree $D_{\mathcal{T}}$ is bounded by:

$$w(\mathcal{T}) \cdot c(S_{\mathcal{T}}) \leq w(\mathcal{T}) \cdot c(S^*_{\mathcal{T}})$$
$$\leq \frac{2}{1-\delta} \cdot \sum_{k=\frac{1+\delta}{2} \cdot w(\mathcal{T})+1}^{w(\mathcal{T})} c\left(S^*_{\lfloor k/2 \rfloor} \cap \mathcal{T}\right).$$

Sketch of the proof

- Let $\beta = \frac{1-\delta}{2}$. We have:

$$c_T(D) \leq \frac{2}{1-\delta} \cdot \sum_{\mathcal{T}} \sum_{k=\frac{1+\delta}{2} \cdot w(\mathcal{T})+1}^{w(\mathcal{T})} c\left(S^*_{\lfloor k/2 \rfloor} \cap \mathcal{T}\right)$$

$$\leq \frac{2}{1-\delta} \cdot \sum_{k=0}^{w(T)} c\left(S^*_{\lfloor k/2 \rfloor}\right) \leq (4+\epsilon) \cdot \mathtt{OPT}(T)\,.$$

Sketch of the proof

- Let $\beta = \frac{1-\delta}{2}$. We have:

$$c_T(D) \leq \frac{2}{1-\delta} \cdot \sum_{\mathcal{T}} \sum_{k=\frac{1+\delta}{2} \cdot w(\mathcal{T})+1}^{w(\mathcal{T})} c\left(S^*_{\lfloor k/2 \rfloor} \cap \mathcal{T}\right)$$

$$\leq \frac{2}{1-\delta} \cdot \sum_{k=0}^{w(T)} c\left(S^*_{\lfloor k/2 \rfloor}\right) \leq (4+\epsilon) \cdot \mathtt{OPT}(T)\,.$$

- As $1/\delta = \frac{4+\epsilon}{\epsilon} = 1 + 4/\epsilon$, the running time is $O\left(n^4/\epsilon^2\right)$.

Min–Ratio Vertex Cut Problem

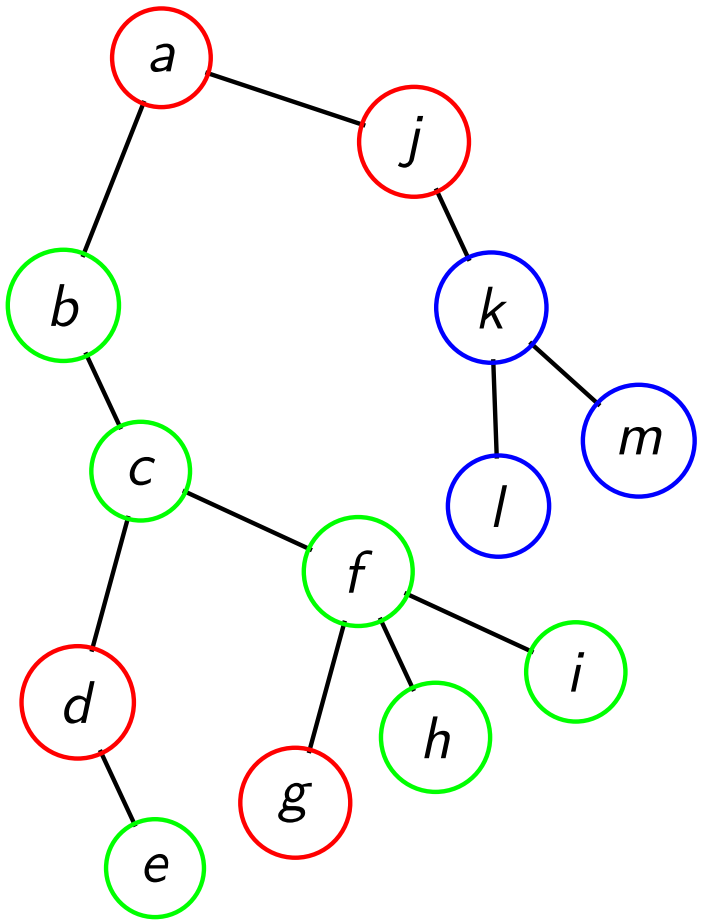
Min–Ratio Vertex Cut Problem

Input: Graph $G = (V(G), E(G))$, the cost function $c\colon V \to \mathbb{N}$ and the weight function $w\colon V \to \mathbb{N}$.

Output: A partition (A, S, B) of $V(G)$ called **vertex–cut**, such that there are no $u \in A$ and $v \in B$ for which $uv \in E(G)$, minimizing the ratio:

$$\alpha_{c,w}(A, S, B) = \frac{c(S)}{w(A \cup S) \cdot w(B \cup S)}.$$

Example of a vertex cut



	$w(v)$	$c(v)$
a	2	3
b	1	4
c	3	6
d	2	2
e	4	1
f	0	3
g	1	1
h	4	3
i	2	3
j	5	2
k	1	2
l	2	3
m	3	4

Figure: A tree and a vertex cut of ratio $\alpha_{c,w}(A, S, B) = \frac{c(S)}{w(A \cup S) \cdot w(B \cup S)} = \frac{8}{(6+10) \cdot (14+10)} = \frac{8}{384} = \frac{1}{48}$.

How to find the cut

We use the following result of [FHL05]:

Theorem

Given a graph $G = (V(G), E(G))$, *the cost function* $c\colon V \to \mathbb{N}$ *and the weight function* $w\colon V \to \mathbb{N}$, *there exists a polynomial–time algorithm, which computes a partition* (A, S, B), *such that:*

$$\alpha_{c,w}(A, S, B) = O\left(\sqrt{\log n}\right) \cdot \alpha_{c,w}(G)\,.$$

Searching in Graphs

We will iteratively use the f_n–approximation algorithm for the Min–Ratio Vertex Cut Problem to create an $O(f_n)$–approximation algorithm for the Graph Search Problem:

Theorem

Let f_n be the approximation ratio of any polynomial time algorithm for the Min–Ratio Vertex Cut Problem. Then, there exists an $O(f_n)$–approximation algorithm for the Graph Search Problem, running in polynomial time.

The algorithm

proc `DecisionTree`(T, c, w, ϵ):

1. $A_G, S_G, B_G \leftarrow$ `AlgorithmMinCut`(G, c, w).
2. $D_G \leftarrow$ arbitrary partial decision tree for G, built from vertices of S_G.
3. For each $H \in G - S_G$:
   - 3.1 $D_H \leftarrow$ `DecisionTree`(H, c, w).
   - 3.2 Hang D_H in D_G below the last query to $v \in N_G(H)$.
4. Return D_G.

Sketch of the proof

- $\mathcal{G}$ – any subgraph of G, for which the procedure was called.
- Let $S^*_{\mathcal{G}} = S^*_{\lfloor w(\mathcal{G})/2 \rfloor} \cap \mathcal{G}$.

Sketch of the proof

- $\mathcal{G}$ – any subgraph of G, for which the procedure was called.
- Let $S^*_{\mathcal{G}} = S^*_{\lfloor w(\mathcal{G})/2 \rfloor} \cap \mathcal{G}$.

Lemma

Let $\mathcal{H} = \mathcal{G} - S^*_{\mathcal{G}}$. Then, we can partition $\mathcal{H}$ into two sets, $\mathcal{A}$ and $\mathcal{B}$ such that for $A = \bigcup_{H \in \mathcal{A}} V(H)$ and $B = \bigcup_{H \in \mathcal{B}} V(H)$, we have:

$$w\left(A \cup S^*_{\mathcal{G}}\right) \cdot w\left(B \cup S^*_{\mathcal{G}}\right) \geq w(\mathcal{G})^2 / 11.$$

The partition $\left(A, S^*_{\mathcal{G}}, B\right)$ in the above lemma is a vertex cut of $\mathcal{G}$, so we have:

$$\alpha_{c,w}(\mathcal{G}) \leq \frac{c\left(S^*_{\mathcal{G}}\right)}{w\left(A \cup S^*_{\mathcal{G}}\right) \cdot w\left(B \cup S^*_{\mathcal{G}}\right)} \leq \frac{11 \cdot c\left(S^*_{\mathcal{G}}\right)}{w(\mathcal{G})^2}.$$

Sketch of the proof

- Let $(A_{\mathcal{G}}, S_{\mathcal{G}}, B_{\mathcal{G}})$, be the partition returned by the algorithm:

$$\alpha_{c,w}(A_{\mathcal{G}}, S_{\mathcal{G}}, B_{\mathcal{G}}) = \frac{c(S_{\mathcal{G}})}{w(A_{\mathcal{G}} \cup S_{\mathcal{G}}) \cdot w(B_{\mathcal{G}} \cup S_{\mathcal{G}})} \leq f_n \cdot \frac{11 \cdot c\left(S^*_{\mathcal{G}}\right)}{w(\mathcal{G})^2}.$$

- Let $\beta = w(B_{\mathcal{G}} \cup S_{\mathcal{G}}) / w(\mathcal{G})$, so that $(1 - \beta) \cdot w(\mathcal{G}) = w(A_{\mathcal{G}})$.
- Assume w. l. o. s. that $w(A_{\mathcal{G}}) \geq w(B_{\mathcal{G}})$.

Sketch of the proof

- Let $(A_{\mathcal{G}}, S_{\mathcal{G}}, B_{\mathcal{G}})$, be the partition returned by the algorithm:

$$\alpha_{c,w}(A_{\mathcal{G}}, S_{\mathcal{G}}, B_{\mathcal{G}}) = \frac{c(S_{\mathcal{G}})}{w(A_{\mathcal{G}} \cup S_{\mathcal{G}}) \cdot w(B_{\mathcal{G}} \cup S_{\mathcal{G}})} \leq f_n \cdot \frac{11 \cdot c\left(S^*_{\mathcal{G}}\right)}{w(\mathcal{G})^2}.$$

- Let $\beta = w(B_{\mathcal{G}} \cup S_{\mathcal{G}}) / w(\mathcal{G})$, so that $(1 - \beta) \cdot w(\mathcal{G}) = w(A_{\mathcal{G}})$.
- Assume w. l. o. s. that $w(A_{\mathcal{G}}) \geq w(B_{\mathcal{G}})$.
- The contribution of the decision tree $D_{\mathcal{G}}$ is bounded by:

$$\begin{aligned} w(\mathcal{G}) \cdot c(S_{\mathcal{G}}) &\leq 11 \cdot f_n \cdot \frac{w(A_{\mathcal{G}} \cup S_{\mathcal{G}}) \cdot w(B_{\mathcal{G}} \cup S_{\mathcal{G}})}{w(\mathcal{G})} \cdot c\left(S^*_{\mathcal{G}}\right) \\ &\leq 11 \cdot f_n \cdot w(B_{\mathcal{G}} \cup S_{\mathcal{G}}) \cdot c\left(S^*_{\mathcal{G}}\right) \\ &\leq 11 \cdot f_n \cdot \sum_{k=w(A_{\mathcal{G}})+1}^{w(\mathcal{G})} c\left(S^*_{\lfloor k/2 \rfloor} \cap \mathcal{G}\right) \end{aligned}$$

Sketch of the proof

Let D be the decision tree returned by the procedure. We have:

$$
\begin{aligned}
c_G(D) &\leq \sum_{\mathcal{G}} w(\mathcal{G}) \cdot c(S_{\mathcal{G}}) \\
&\leq 11 \cdot f_n \cdot \sum_{\mathcal{G}} \sum_{k=w(A_{\mathcal{G}})+1}^{w(\mathcal{G})} c\left(S^*_{\lfloor k/2 \rfloor} \cap \mathcal{G}\right) \\
&\leq 11 \cdot f_n \cdot \sum_{k=0}^{w(G)} c\left(S^*_{\lfloor k/2 \rfloor}\right) \\
&\leq 22 \cdot f_n \cdot \mathtt{OPT}(G)
\end{aligned}
$$

Summary

- The key idea behind our lower bounds is to decompose the optimal decision tree (and its cost) into a series of separators.

Summary

- The key idea behind our lower bounds is to decompose the optimal decision tree (and its cost) into a series of separators.
- This immediately yields approximation algorithms for the Graph Search Problem: $(4+\epsilon)$ for trees and $O\left(\sqrt{\log n}\right)$ for general graphs.

Summary

- The key idea behind our lower bounds is to decompose the optimal decision tree (and its cost) into a series of separators.
- This immediately yields approximation algorithms for the Graph Search Problem: $(4+\epsilon)$ for trees and $O\left(\sqrt{\log n}\right)$ for general graphs.
- Currently, we are working on an FPTAS parametrized by the number of non-leaf vertices of the graph as well as a more general model in which the cost of a query depends on the target vertex.

Bibliography I

[FHL05] Uriel Feige, MohammadTaghi Hajiaghayi, and James R. Lee. "Improved approximation algorithms for minimum-weight vertex separators". In: *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*. STOC '05. Baltimore, MD, USA: Association for Computing Machinery, 2005, pp. 563–572. ISBN: 1581139608. DOI: 10.1145/1060590.1060674. URL: https://doi.org/10.1145/1060590.1060674.